Indexes & Performance

Write Queries That Scale to Billions of Rows

LEARNING OUTCOME

By the end of this module you will understand how indexes work, create the right indexes for your queries, read EXPLAIN plans to diagnose slow queries, and apply query optimisation techniques that make queries run faster.

Skills You Will Gain
• Understand B-tree indexes and how they speed up queries
• Create single-column, composite, and partial indexes
• Read EXPLAIN and EXPLAIN ANALYZE output
• Identify full table scans and how to fix them
• Understand index selectivity and cardinality
• Apply query rewriting for better performance

■ A query that takes 30 seconds without an index can take 10ms with one. Index design is the single highest-leverage skill for database performance. It separates junior from senior engineers.

SECTION 1

How Indexes Work

Index = Sorted Copy of Data with Pointers

Without index: database reads EVERY row (full table scan) -- O(n) With index: database jumps to relevant rows via sorted structure -- O(log n) B-tree index (default): balanced tree sorted by indexed column. - Great for: =, <, >, BETWEEN, ORDER BY on indexed column - Not useful: LIKE '%pattern%' (leading wildcard), functions on column Index cost: extra disk space + slower INSERTs/UPDATEs/DELETEs. Rule of thumb: index columns used in WHERE, JOIN ON, ORDER BY, GROUP BY.

```
-- Create a simple B-tree index CREATE INDEX idx_customers_city ON customers(city); CREATE INDEX
idx_orders_customer_id ON orders(customer_id); CREATE INDEX idx_orders_date ON orders(order_date);
-- UNIQUE index: enforces uniqueness + fast lookup CREATE UNIQUE INDEX idx_customers_email ON
customers(email); -- COMPOSITE index: covers multiple columns -- Column order matters! Most
selective column first, -- then columns used in range conditions last CREATE INDEX
idx_orders_customer_date ON orders(customer_id, order_date); CREATE INDEX idx_orders_status_date ON
orders(status, order_date DESC); -- PARTIAL index: only index subset of rows (PostgreSQL) CREATE
INDEX idx_orders_active ON orders(customer_id) WHERE status NOT IN ('cancelled', 'refunded'); --
Smaller index, faster for queries that filter active orders -- COVERING index: include all columns
the query needs CREATE INDEX idx_covering ON orders(customer_id, order_date) INCLUDE (total_amount,
status); -- PostgreSQL INCLUDE syntax -- Query can be answered entirely from index -- no table
access! -- DROP index DROP INDEX idx_customers_city; -- View existing indexes SHOW INDEX FROM
orders; -- MySQL SELECT * FROM pg_indexes WHERE tablename = 'orders'; -- PostgreSQL --
Rebuild/analyze index stats ANALYZE orders; -- PostgreSQL update statistics OPTIMIZE TABLE orders;
-- MySQL rebuild
```

SECTION 2

Reading EXPLAIN Plans

```
-- EXPLAIN shows the query execution plan (free -- no execution) EXPLAIN SELECT * FROM orders WHERE
customer_id = 42; -- Good output: 'Index Scan using idx_orders_customer_id' -- Bad output: 'Seq
Scan on orders' (full table scan) -- EXPLAIN ANALYZE actually runs the query and shows real timing
EXPLAIN ANALYZE SELECT o.order_id, c.first_name, SUM(oi.quantity) AS items FROM orders o JOIN
customers c ON o.customer_id = c.customer_id JOIN order_items oi ON o.order_id = oi.order_id WHERE
o.status = 'delivered' AND o.order_date >= '2024-01-01' GROUP BY o.order_id, c.first_name; --
EXPLAIN output key fields: -- Seq Scan -- full table scan -- INDEX NEEDED if large table -- Index
Scan -- using index to find rows -- Index Only Scan -- even better: answers from index alone --
Nested Loop -- join method for small tables -- Hash Join -- join method for medium tables -- Merge
Join -- join method for large sorted tables -- cost=0.00..1234.56 -- estimated cost (relative
units) -- rows=150000 -- estimated row count -- actual time=0.042..3.521 ms -- real time (ANALYZE
only) -- loops=1 -- how many times this node ran
```

SECTION 3

Query Optimisation Techniques

```
-- ■ SLOW: function on indexed column breaks index usage SELECT * FROM orders WHERE
YEAR(order_date) = 2024; -- ■ FAST: use range query instead SELECT * FROM orders WHERE order_date
>= '2024-01-01' AND order_date < '2025-01-01'; -- ■ SLOW: leading wildcard cannot use index SELECT
* FROM customers WHERE email LIKE '%@gmail.com'; -- ■ FAST: anchor to left for B-tree usage SELECT
* FROM customers WHERE email LIKE 'alice%'; -- For CONTAINS queries use FULLTEXT search -- ■ SLOW:
SELECT * fetches unnecessary columns SELECT * FROM orders WHERE customer_id = 42; -- ■ FAST: select
only needed columns SELECT order_id, order_date, total_amount FROM orders WHERE customer_id = 42;
-- ■ SLOW: correlated subquery runs once per row SELECT c.first_name, (SELECT SUM(total_amount)
```

```
FROM orders o WHERE o.customer_id=c.customer_id) AS total FROM customers c; -- ■ FAST: equivalent with JOIN + GROUP BY SELECT c.first_name, SUM(o.total_amount) AS total FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id GROUP BY c.customer_id, c.first_name; -- ■ SLOW: DISTINCT when JOIN creates duplicates SELECT DISTINCT c.* FROM customers c JOIN orders o ON c.customer_id=o.customer_id; -- ■ FAST: use EXISTS instead SELECT c.* FROM customers c WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id=c.customer_id); -- Index selectivity: high cardinality = better index -- Good index column: email (millions of unique values) -- Poor index column: is_active (only 2 values: true/false) -- not worth indexing -- CHECK INDEX USAGE (MySQL) SHOW STATUS LIKE 'Handler_read%'; -- Handler_read_key high = good index usage -- Find slow queries (MySQL) SET GLOBAL slow_query_log = 'ON'; SET GLOBAL long_query_time = 1; -- log queries taking > 1 second
```

■ PRACTICE Q & A

Q1. What is a full table scan and when is it acceptable?

A: *A full table scan reads every row in the table. Acceptable for small tables (< few thousand rows) or when fetching > ~20% of rows. On large tables (millions of rows), always add an index for filtered columns.*

Q2. What is index cardinality and why does it matter?

A: *Cardinality = number of distinct values. High cardinality (email, order_id) = excellent for indexing. Low cardinality (boolean, status with 3 values) = poor index -- database may prefer full scan.*

Q3. What is a composite index and what is the column order rule?

A: *An index on multiple columns. The leftmost prefix rule: a composite index on (a, b, c) helps queries filtering on a, or a+b, or a+b+c -- but NOT queries filtering only on b or c.*

Q4. Why does applying a function to an indexed column break the index?

A: *The index stores raw column values. When you write WHERE YEAR(date) = 2024, the database cannot look up 'YEAR(date)=2024' in the index -- it has to calculate YEAR for every row. Use range conditions instead.*

Q5. What is a covering index?

A: *An index that contains ALL columns the query needs. The database can answer the query from the index alone without touching the base table -- called an 'index-only scan'. Fastest possible read.*

■ Indexes & Performance Cheat Sheet	
`CREATE INDEX idx_name ON tbl(col)`	Create B-tree index
`CREATE UNIQUE INDEX ... ON tbl(col)`	Unique index
`CREATE INDEX ... ON tbl(col1,col2)`	Composite index
`EXPLAIN SELECT ...`	Show execution plan (no run)
`EXPLAIN ANALYZE SELECT ...`	Show plan with real timing
`Seq Scan = full table scan`	Needs index if table is large
`Index Scan = index used`	Good
`Index Only Scan = covering`	Best -- no table access
`WHERE date >= '2024-01-01'`	Range not function on col
`EXISTS over DISTINCT`	Better for join dedup
`High cardinality = good index`	email > status > boolean
`ANALYZE tbl`	Update statistics for planner